# Aterrizaje lunar con Deep Q-Networks: implementación, análisis de la política y estudio de ablación

Thiago Rivas
C.I. 5.475.680-3
*Facultad de Ingeniería*
*Universidad de la República*
Montevideo, Uruguay
thiago.rivas0021@gmail.com

Juan Duarte
C.I. 5.201.302-9
*Facultad de Ingeniería*
*Universidad de la República*
Montevideo, Uruguay
juan.duarte@fing.edu.uy

Agustin Occhiuzzi
C.I. 5.480.562-4
*Facultad de Ingeniería*
*Universidad de la República*
Montevideo, Uruguay
agustin.occhiuzzi@fing.edu.uy

***Resumen*. Se implementó un agente DQN para LunarLander-v3 y se estudió cómo aprende, qué política obtiene y qué pasa al quitarle tres partes al algoritmo. Después de algunas pruebas de calibración se eligió una red con dos capas ocultas de 64 neuronas y una tasa de aprendizaje de $10^{-3}$. El estudio final tiene 32 agentes, ocho por condición, entrenados durante 300 episodios cada uno. El agente completo obtuvo un retorno medio de 137,8 y logró un retorno de al menos 200 en el 40,44 % de los episodios, mientras que el agente aleatorio no lo logró en ninguno. Los cortes de la política muestran que el agente aprendió a frenar la caída y a corregir el giro. Para comparar las variantes se usó el test de Mann–Whitney con la corrección de Holm, y solo quitar el replay dio una diferencia significativa. Estos resultados tienen límites: se usaron máquinas distintas, se evaluó en escenarios ya usados en la calibración y se usaron pocas semillas.**

***Palabras clave*. aprendizaje por refuerzos, DQN, LunarLander, ablación**

## I. Introducción

En LunarLander-v3 hay que aterrizar una nave usando un motor principal y dos motores laterales. Se usó la versión discreta de Gymnasium [1], sin viento. El estado es $s = (x, y, v_x, v_y, \theta, \omega, c_1, c_2)$: posición, velocidad, ángulo, velocidad angular y dos indicadores que dicen si cada pata toca el suelo. Hay cuatro acciones: no hacer nada, encender el motor izquierdo, encender el principal o encender el derecho.

El objetivo fue entrenar un agente con DQN [2] y ver qué aportan la memoria de repetición, la red objetivo y la exploración. Se usó el criterio de la tarea: un episodio es exitoso si su retorno total $R = \sum_t r_t$ es de al menos 200, y la tasa de éxito tiene que ser mayor que 10 %. Este criterio mira solo la recompensa, así que no comprueba directamente que la nave haya aterrizado.

## II. Solución DQN implementada

La clase `AgenteDQN` usa una red neuronal $Q_\theta$ que, dado un estado, estima cuánto vale cada acción. La red recibe los ocho valores del estado, tiene dos capas ocultas con ReLU y devuelve cuatro números, uno por acción. Cada número estima el retorno descontado que se espera obtener si se toma esa acción [3]. Si desde el paso $t$ se reciben las recompensas $r_t, r_{t+1}, \ldots$, ese retorno es $G_t = \sum_{k\geq 0} \gamma^k r_{t+k}$, o sea que cada recompensa se multiplica por $\gamma$ una vez por cada paso que falta para recibirla. Con $\gamma = 0{,}99$, una recompensa que llega 100 pasos después todavía vale cerca del 37 % ($0{,}99^{100} \approx 0{,}37$). Por eso el agente tiene en cuenta cosas que pasan bastante más adelante, como el premio por aterrizar. La red tiene 4996 parámetros. El DQN original trabajaba con imágenes de juegos de Atari [2]. Acá, en cambio, la red recibe directamente el vector de estado y usa solo capas completamente conectadas. No se normalizan las observaciones ni se recortan las recompensas.

Cada transición $(s, a, r, s', d)$ se guarda en una memoria de repetición (*replay*) con capacidad fija. Cada vez que se actualiza la red, se sacan al azar 64 transiciones distintas de esa memoria (un minilote). Así se aprovecha cada experiencia más de una vez y se evita aprender solo de transiciones seguidas. Esto importa porque de un paso al siguiente la nave está casi en el mismo lugar. Si la red aprendiera de las transiciones en orden, muchas actualizaciones seguidas la empujarían hacia el mismo lado: se adaptaría a la situación del momento y podría olvidar lo que aprendió antes.

Además se usa una segunda red, la red objetivo $Q_{\theta^-}$. Empieza siendo una copia de $Q_\theta$, no se entrena y cada cierto número de pasos se vuelve a copiar. Con ella se calcula el valor objetivo de cada transición:

$$z = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a'). \tag{1}$$

En $z$, $r$ es la recompensa que realmente se recibió. La parte $\max_{a'} Q_{\theta^-}(s', a')$ es lo que la red estima que se puede ganar desde $s'$ eligiendo la mejor acción, y $\gamma$ lo descuenta. El factor $(1 - d)$ vale 0 cuando el episodio terminó. En ese caso el objetivo es solo $r$, porque después del final ya no hay más recompensas.

Si el objetivo se calculara con la misma red que se está entrenando, cada vez que la red cambia también cambiaría el valor $z$ al que se quiere llegar, como un perro que se persigue

Tabla I
CONFIGURACIÓN DE REFERENCIA A0

| Parámetro | Valor |
|---|---|
| Red Q | $8 \to 64 \to 64 \to 4$, ReLU |
| Adam | $\eta = 10^{-3}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ |
| Descuento | $\gamma = 0{,}99$ |
| Memoria / minilote | 50 000 / 64 transiciones |
| Inicio / frecuencia | 1000 pasos / cada 4 pasos |
| Copia a red objetivo | Cada 1000 pasos de entorno |
| Exploración | Lineal por episodio, $1 \to 0{,}05$ |
| Pérdida / gradiente | Huber ($\beta = 1$) / norma $\leq 10$ |
| Presupuesto | 300 episodios, hasta 1000 pasos |

la cola. Con la red objetivo fija entre copias, el objetivo se queda quieto un tiempo y el entrenamiento es más estable.

La red se entrena para que su predicción $Q_\theta(s, a)$ se acerque a $z$, usando la pérdida de Huber (SmoothL1 en PyTorch) promediada sobre el minilote. El valor $z$ se toma como un número fijo, sin calcular gradientes a través de él. Los pesos se ajustan con Adam, y se pone un límite al tamaño del gradiente para que ninguna actualización sea demasiado grande.

La variable $d$ vale 1 solo cuando el episodio termina de verdad (`terminated`). Si el episodio se corta porque llegó al límite de 1000 pasos (truncamiento), se sigue usando el valor futuro, porque la nave no llegó a un final real. Si ese corte se tratara como un final, la red aprendería que en esos estados no queda nada por ganar, aunque la nave podría haber seguido volando y aterrizado. Además, como el estado no incluye el tiempo, la red no podría distinguir esos estados de otros iguales que no llegaron al límite.

Durante el entrenamiento se usa la estrategia $\varepsilon$-greedy: con probabilidad $\varepsilon$ se elige una acción al azar y, si no, la de mayor valor. Después de $e$ episodios, $\varepsilon$ vale

$$\varepsilon_e = \max\left(0{,}05,\ 1 - 0{,}95\frac{e}{200}\right). \tag{2}$$

Al principio la red todavía no aprendió nada y sus valores son casi al azar, así que conviene probar muchas acciones para descubrir cuáles dan buenas recompensas. A medida que aprende, conviene usar más lo que ya sabe. Desde el episodio 200 queda un 5 % de acciones al azar, para seguir probando un poco.

En la evaluación siempre se elige la acción de mayor valor, $\arg\max_a Q_\theta(s, a)$, sin explorar y sin cambiar los pesos.

La tabla I resume la configuración. Siempre se entrenan los 300 episodios completos y se evalúan los pesos del final, sin quedarse con el mejor momento del entrenamiento.

## III. SELECCIÓN DE HIPERPARÁMETROS

La calibración tuvo tres partes: arquitectura y tasa de aprendizaje, tamaño de la memoria y forma de explorar. Para elegir la configuración se usaron los resultados guardados de esas pruebas, que no siempre tuvieron el mismo presupuesto ni la misma configuración.

Para la arquitectura y la tasa se entrenaron A0, A1 y A2 con las semillas 42 y 1042, durante 300 episodios, y cada agente se evaluó en 200 escenarios. A1 cambia a la vez el tamaño de las capas y la tasa respecto de A0. Por eso se agregó A2: entre A0 y A2 cambia solo la tasa, y entre A2 y A1 cambia solo el tamaño. Después se probó A3, igual a A0 pero con una tasa más alta. La tabla II muestra el retorno medio de cada agente.

Tabla II
CALIBRACIÓN DE ARQUITECTURA Y TASA: RETORNO MEDIO EN 200 EPISODIOS POR AGENTE

| Config. | Capas ocultas | $\eta$ | Semilla 42 | Semilla 1042 |
|---|---|---|---|---|
| A0 | 64, 64 | $10^{-3}$ | 167,2 | 209,8 |
| A1 | 128, 128 | $5 \cdot 10^{-4}$ | 142,1 | 33,7 |
| A2 | 64, 64 | $5 \cdot 10^{-4}$ | −7,4 | 7,7 |
| A3 | 64, 64 | $5 \cdot 10^{-3}$ | 203,3 | 99,9 |

Promediando las dos semillas, A0 obtuvo 188,5, A1 87,9, A2 0,1 y A3 151,6. Por eso se eligió A0, aunque A3 le gana en la semilla 42. Con solo dos entrenamientos por configuración no se puede decir que A0 sea la mejor en general.

En una prueba anterior se compararon memorias de 100 000 y 10 000 transiciones y un prototipo sin memoria. Con la semilla 42, 500 episodios de entrenamiento y 50 de evaluación, los retornos medios fueron 247,7, 223,6 y −87,7, y las tasas de éxito, 86 %, 80 % y 0 %. Ese agente tenía capas de 128 neuronas y $\eta = 5 \cdot 10^{-4}$. Los valores que usa A0 (memoria de 50 000 y minilote de 64) no se eligieron con una búsqueda sistemática.

La prueba anterior de exploración usó cinco semillas, 600 episodios de entrenamiento y 100 de evaluación por agente, con una red 128/128, $\varepsilon$ mínimo de 0,01 y actualización suave de la red objetivo. En la actualización suave, en vez de copiar todos los pesos de golpe cada cierto tiempo, la red objetivo se acerca un poco a la que aprende en cada actualización: $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$, con $\tau$ chico. Así el objetivo cambia de a poco en lugar de dar saltos. Bajar $\varepsilon$ de forma exponencial por episodios dio un retorno medio de 177,1, y bajarlo de forma lineal en 50 000 pasos dio 127,1. El test de Mann–Whitney bilateral dio $p = 0{,}2222$, sin corregir por comparaciones múltiples. El estudio final usó la bajada lineal por episodios de A0, que es distinta de las dos.

## IV. APRENDIZAJE Y DESEMPEÑO FINAL

### *IV-A. Selección y evaluación*

El estudio final tiene ocho agentes por condición: completo, sin replay, sin red objetivo y sin exploración. Hay tres agentes completos más que quedaron fuera de los promedios y de los tests. Qué corridas entraron se decidió por su origen y por las semillas que cubrían, no por lo bien que les fue.

Las semillas de entrenamiento son 42, 1042, 2042, 3042, 4042, 5042, 6042 y 7042. Se fijan las semillas de Python, NumPy, PyTorch y del espacio de acciones. El episodio $i$ (contando desde cero) empieza con la semilla de entrenamiento más $i$. Cada agente final se evalúa sin explorar en los escenarios 10000 a 10199, que son los mismos que se usaron en la calibración. Fijar las semillas permite repetir exactamente un entrenamiento en la misma máquina y probar todas las condiciones en los mismos escenarios. Hacen falta

Tabla III
EVALUACIÓN DEL ESTUDIO FINAL POR SEMILLA DE ENTRENAMIENTO

| Semilla | Completo | Sin objetivo | Sin replay | Sin explor. |
|---|---|---|---|---|
| 42 | 174,9 | 140,3 | −28,2 | −168,7 |
| 1042 | 183,2 | −211,9 | 122,8 | 62,6 |
| 2042 | 80,6 | 189,3 | 6,2 | 150,7 |
| 3042 | 69,5 | 20,6 | −129,5 | −70,2 |
| 4042 | 60,5 | 0,7 | −129,2 | −133,3 |
| 5042 | 121,9 | 81,7 | 107,6 | 65,9 |
| 6042 | 203,2 | 149,7 | 38,3 | 119,2 |
| 7042 | 208,7 | 7,0 | 5,1 | 180,3 |
| Media | 137,8 | 47,2 | −0,8 | 25,8 |
| DE entre semillas | 62,0 | 126,6 | 94,5 | 132,8 |
| Éxitos / 1600 | 647 | 352 | 189 | 287 |
| Éxito (%) | 40,44 | 22,00 | 11,81 | 17,94 |

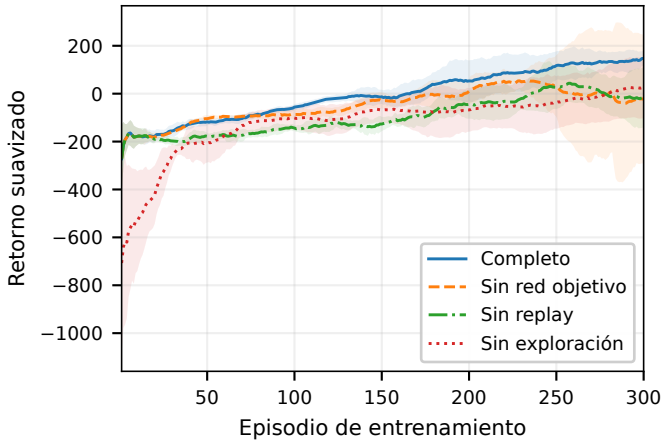



Figura 1. Retorno durante el entrenamiento, suavizado con el promedio de los últimos 20 episodios (o menos, al principio). Cada línea es el promedio de ocho semillas. Las bandas muestran ±1 DE entre semillas y no son intervalos de confianza.

varias semillas porque el resultado de DQN depende mucho de la suerte: en la tabla III, el agente completo va de 60,5 a 208,7 según la semilla.

Cada agente entrenado cuenta como un solo dato. Sus 200 episodios de evaluación sirven para medir qué tan bueno es ese agente, pero no son 200 entrenamientos distintos. Todos vienen del mismo entrenamiento: si una semilla dio una red mala, sus 200 episodios van a ser malos juntos. Como lo que se compara es la forma de entrenar, contarlos como 200 datos haría que las diferencias parecieran mucho más seguras de lo que son. En la tabla III, cada celda es el promedio de esos 200 retornos. Las últimas filas muestran el promedio de las ocho semillas y su desviación estándar (DE). Como todos los agentes tienen 200 evaluaciones, la tasa de éxito es la cantidad de éxitos sobre los 1600 episodios.

### *IV-B. Curvas y comparación con el azar*

La figura 1 muestra que el agente completo mejora: sus primeros 20 episodios promedian −177,6 y los últimos 20, 147,0. Las ablaciones también mejoran, pero quedan por debajo del completo. Sin red objetivo, al final las semillas dan resultados muy distintos entre sí.

El agente completo tuvo 647 éxitos en 1600 evaluaciones (40,44 %). Siete de los ocho agentes superan el 10 %. La excepción es la semilla 3042, con 7 éxitos de 200 (3,5 %). Las tasas van de 3,5 % a 71,5 %, así que cumplir el criterio en promedio no garantiza cumplirlo en todos los entrenamientos.

Para comparar con un agente que elige al azar en la misma máquina, se usó el notebook final. Ahí se volvió a evaluar el agente completo de semilla 42 en 200 episodios: obtuvo 168,86 ± 81,11 de retorno y 63 éxitos de 200 (31,5 %). El agente aleatorio, con su generador fijado en 42 y en los mismos escenarios, obtuvo −183,17±111,96 y ningún éxito. Acá ± es la desviación estándar de los 200 retornos. Esta evaluación da un poco distinto que la original de esa red (174,90 y 68/200), así que no se usó para reemplazar los datos del estudio.

## V. ANÁLISIS DE LA POLÍTICA

Para ver qué aprendió el agente se hicieron tres cortes. En cada uno se parte de un estado de referencia y se mueven solo dos de sus valores en una grilla de $101 \times 101$: $(x, v_x)$, $(\theta, \omega)$ o $(y, v_y)$. Los otros seis quedan fijos. Esto hace falta porque la política depende de ocho variables y no se puede dibujar entera. Fijando seis, la acción depende solo de dos y se puede mostrar como una imagen, aunque cada corte muestra solo una parte de la política. El estado de referencia tiene $x = v_x = \theta = \omega = c_1 = c_2 = 0$, $y = 0{,}8$ y $v_y = -0{,}3$. Se eligió la semilla 42 por ser la primera de la lista, no por su resultado. La figura 2 muestra, en cada punto, la acción que elige la red, calculada con los pesos guardados.

En el corte vertical aparece una frontera inclinada: el motor principal se enciende cuando la nave cae rápido, y cerca del suelo también cuando cae más despacio. Esto tiene sentido, porque el motor no puede frenar la nave de golpe: cuanto más rápido cae, más altura necesita para frenar. Por eso, si cae rápido conviene empezar a frenar antes, y si cae despacio alcanza con frenar cerca del suelo. Más arriba de esa zona aparecen tanto no hacer nada como el motor derecho, así que el agente no sigue simplemente la regla de frenar o no frenar.

En el corte del ángulo, a cada lado de una frontera que depende de $\theta$ y $\omega$ se usa un motor lateral distinto. Cerca de esa frontera, el motor derecho reduce el ángulo y el izquierdo lo aumenta. Parece que el agente corrige a la vez la inclinación y el giro, aunque esto no alcanza para decir que siga una regla lineal en todos los estados. En los cortes de las ocho semillas, guardados en el notebook, se ve algo parecido, aunque cambian las fronteras y las zonas donde no hace nada o usa el motor principal.

En el corte horizontal, si la nave se mueve rápido hacia la derecha ($v_x$ positiva y alta), el agente prefiere el motor izquierdo, lo que parece servir para frenar ese desplazamiento. La división depende también de $x$ y no es simétrica. Con un solo estado de referencia no se puede decir que el agente prefiera un lado en general.

Para ver cómo terminan los episodios, las 32 redes se evaluaron de nuevo en 20 escenarios cada una (10000 a 10019). Con eso se armó la tabla IV. Un final cuenta como

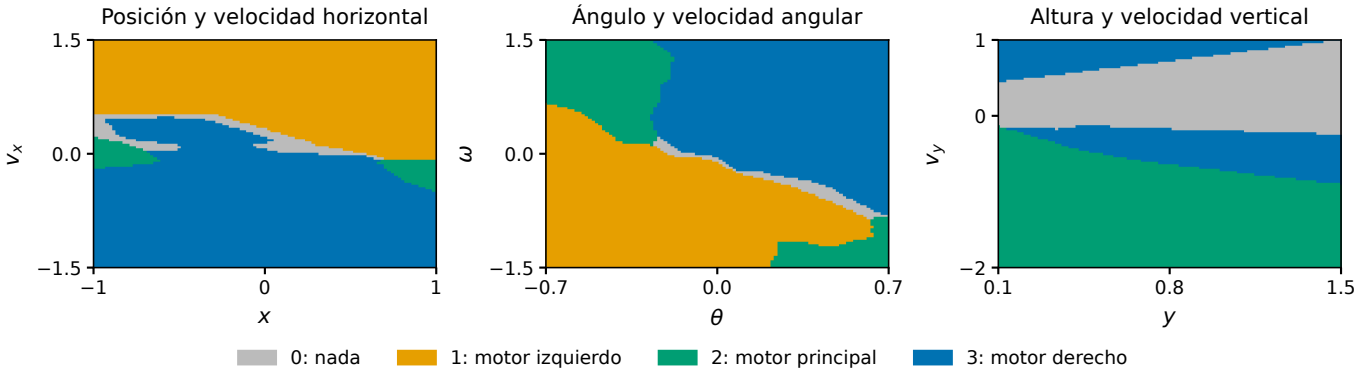



Figura 2. DQN completo, semilla 42: acción que elige la red en cada punto. En cada panel solo cambian las dos variables de los ejes. Las demás quedan como en el estado de referencia.

Tabla IV
CÓMO TERMINAN LOS EPISODIOS EN LA REEVALUACIÓN HECHA EN MACOS (160 EPISODIOS POR CONDICIÓN)

| Condición | Posado | Choque/salida | Truncado |
|---|---|---|---|
| Completo | 105 | 7 | 48 |
| Sin red objetivo | 66 | 54 | 40 |
| Sin replay | 22 | 135 | 3 |
| Sin exploración | 35 | 48 | 77 |

Tabla V
COMPARACIONES CONTRA EL AGENTE COMPLETO (OCHO AGENTES POR CONDICIÓN)

| Ablación | $\Delta$ | U | $p$ MW | $p$ Holm |
|---|---|---|---|---|
| Sin objetivo | 90,6 | 47 | 0,1304 | 0,1304 |
| Sin replay | 138,7 | 57 | 0,0070 | 0,0210 |
| Sin explor. | 112,0 | 50 | 0,0650 | 0,1299 |

$\Delta$: retorno medio del agente completo menos el de la ablación. U corresponde al agente completo.

posado si el episodio termina con una recompensa de $+100$ en el último paso. Los demás finales cuentan como choque o salida de la zona, y los truncados son los que llegan a los 1000 pasos.

En el agente completo, la mayoría de los episodios que no terminan posados son truncados (48 de 55). Sin replay, en cambio, casi todos son choques o salidas (135 de 160). Estos números describen el comportamiento, pero no son tasas de éxito. Por ejemplo, el agente completo tiene 105 finales posados, pero solo 68 episodios con $R \geq 200$. Una nave que aterriza gastando mucho combustible puede no llegar a 200.

## VI. ESTUDIO DE ABLACIÓN

### *VI-A. Variantes y test estadístico*

En cada condición se quita una sola parte. Sin red objetivo, el valor futuro se calcula con la misma red que se está entrenando, y $z$ se sigue tomando como un número fijo. Sin replay, cada actualización usa solo la transición que acaba de pasar. Como se mantienen el inicio en el paso 1000 y la actualización cada cuatro pasos, las otras tres transiciones no se usan para aprender. Sin exploración, $\varepsilon = 0$ desde el principio, aunque sigue habiendo azar en los pesos iniciales y en el entorno.

Por lo visto en la sección II, sin red objetivo se espera un entrenamiento más inestable, porque el valor al que se quiere llegar cambia en cada actualización. Sin replay se espera que la red aprenda de datos muy parecidos entre sí, use cada transición como mucho una vez y se olvide de lo que aprendió. Sin exploración se espera que el agente repita lo que su red inicial cree mejor y tarde más en descubrir acciones útiles, aunque el azar del entorno y de los pesos le da algo de variedad.

Para comparar cada variante con el agente completo se usó el test U de Mann–Whitney bilateral exacto [4], con las ocho medias de cada condición. Este test no necesita que los datos sigan una distribución normal, y compara si los dos grupos vienen de la misma distribución, no solo sus medias. Como se hacen tres comparaciones, los valores $p$ se corrigen con el método de Holm [5], con un nivel de significación de $\alpha = 0{,}05$ para el conjunto.

La hipótesis nula es que quitar esa parte no cambia nada, o sea, que las medias de las dos condiciones vienen de la misma distribución. El valor $p$ es la probabilidad de ver una diferencia igual o más grande que la observada si la hipótesis nula fuera cierta. Si $p$ es menor que $\alpha$, se rechaza la hipótesis nula. Cuantos más tests se hacen, más fácil es que alguno dé significativo por casualidad, igual que al tirar muchas monedas es más fácil que alguna salga varias veces seguidas del mismo lado. La corrección de Holm ajusta los valores $p$ para que la chance de encontrar por error una diferencia en alguna de las tres comparaciones siga siendo de 0,05 como máximo. El test supone que los grupos son independientes, lo que hay que tomar con cuidado porque las variantes usan las mismas semillas.

La tabla V muestra que solo la variante sin replay es significativamente distinta del agente completo. El completo le gana en las ocho semillas, por 138,7 puntos en promedio. Esto indica que el replay sirve en esta configuración. Hay que tener

en cuenta que esta variante no solo quita la memoria: también aprende de una sola transición por vez y de menos transiciones en total. Cada actualización depende de una transición muy parecida a las anteriores, así que los ajustes de la red son muy irregulares y la red se adapta a lo último que vio. Esto coincide con la tabla IV: 135 de sus 160 episodios terminan en choque o salida, contra 7 del completo, lo que sugiere que la red no llega a aprender bien a frenar y estabilizarse antes de tocar el suelo.

Sin red objetivo y sin exploración también tienen medias más bajas, pero sus $p$ corregidos son 0,1304 y 0,1299, así que no se puede rechazar la hipótesis nula. Eso no quiere decir que sean iguales al completo ni que esas partes no sirvan. Con ocho semillas y tanta variación, diferencias de 90,6 y 112,0 puntos no alcanzan para ser significativas.

## VII. Limitaciones

Todas las corridas tienen los mismos 300 episodios, pero no la misma cantidad de pasos ni de actualizaciones: usaron entre 32 968 y 241 521 pasos, según cuánto duraron sus episodios. Por eso los resultados se comparan después de 300 episodios, sin decir que el entrenamiento haya terminado de mejorar ni que todas las corridas costaran lo mismo. Además, ocho semillas son pocas para saber bien cuánto varían los resultados. También se usaron máquinas distintas (Windows y macOS) con el mismo código, y no todas las condiciones se corrieron en todas las máquinas. Por eso los tests no pueden separar el efecto de la parte quitada del efecto de la máquina.

La evaluación final usa los mismos escenarios que la calibración, lo que puede favorecer a la configuración elegida. Como no se hizo la evaluación con los escenarios reservados (30000 a 30199), los resultados no muestran cómo le iría al agente en escenarios totalmente nuevos. Las evaluaciones extra sirven para analizar los agentes guardados, pero no cuentan como entrenamientos nuevos. Por último, los cortes muestran solo tres planos, y la conclusión sobre el replay vale para esta forma de quitarlo.

## VIII. Conclusiones

Se implementó un agente DQN que cumple el criterio de éxito de la tarea en promedio y en siete de sus ocho entrenamientos completos. También es mucho mejor que el agente aleatorio. En los cortes se ve que aprendió a frenar y a corregir el ángulo, con algunas asimetrías y diferencias entre semillas.

El resultado más claro del estudio es el del replay: sin él, el agente empeora y la diferencia sigue siendo significativa después de corregir por las tres comparaciones. Con estas corridas no se puede concluir nada firme sobre la red objetivo ni sobre la exploración. Para eso haría falta usar más semillas, correr todas las condiciones en máquinas parecidas y evaluar con los escenarios reservados.

## Uso de Inteligencia Artificial

## Referencias


[1] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2024.

[2] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[3] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018.

[4] H. B. Mann y D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 1947.

[5] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.